

[Volver a 'Parcial 2'](#)

Comenzado el	martes, 13 de julio de 2021, 18:24
Estado	Finalizado
Finalizado en	martes, 13 de julio de 2021, 21:19
Tiempo empleado	2 horas 55 minutos
Puntos	16,00/80,00
Calificación	2,00 de 10,00 (20%)

Pregunta **1**
Finalizado
Puntúa 2,00 sobre 10,00

Dar el tipo y escribir el esquema de recursión primitiva de **TCommand**.
ATENCIÓN: recordar que **NO se puede retroceder** entre preguntas. Una vez que haya aceptado la entrega de esta pregunta, lo escrito se considerará *toda* la entrega, sin posibilidad de revisión. Asegurarse de haber resuelto todo a satisfacción ANTES de avanzar.

```
recTC :: (Distance -> b) -> (Angle -> b) -> (Pen -> b) -> (TCommand -> TCommand -> -> b -> b)
recTC fg ft fp ftc (Go d) = fg d
recTC fg ft fp ftc (Turn a) = ft a
recTC fg ft fp ftc (GrabPen p) = fp p
recTC fg ft fp ftc (t1 :# t2) = ftc (recTC fg ft fp ftc t1) (recTC fg ft fp ftc t2)
```

Comentario:
NOTA: B
OBSERVACIONES:
* Muy bien

Pregunta **2**
Finalizado
Puntúa 2,00 sobre 10,00

Dar el tipo y escribir la función que expresa el esquema de recursión estructural sobre **TCommand**, sin utilizar recursión explícita.
ATENCIÓN: recordar que **NO se puede retroceder** entre preguntas. Una vez que haya aceptado la entrega de esta pregunta, lo escrito se considerará *toda* la entrega, sin posibilidad de revisión. Asegurarse de haber resuelto todo a satisfacción ANTES de avanzar.

```
foldTC :: (Distance -> b) -> (Angle -> b) -> (Pen -> b) -> (b -> b -> b) -> TCommand -> b
foldTC fg ft fp ftc (Go d) = fd d
foldTC fg ft fp ftc (Turn a) = ft a
foldTC fg ft fp ftc (GrabPen p) = fp p
foldTC fg ft fp ftc (t1 :# t2) = ftc (foldTC fg ft fp ftc t1) (foldTC fg ft fp ftc t2)
```

Comentario:
NOTA: R-
OBSERVACIONES:
* Pedía NO utilizar recursión explícita. Debía usarse el esquema del ej. 1

Pregunta **3**

Finalizado

Puntúa 2,00
sobre 10,00

Definir sin usar recursión explícita la función `cantSeq :: TCommand -> Int`, que dado un programa de gráficos de tortuga, describe la cantidad de constructores (`:#:`) del mismo.

ATENCIÓN: recordar que ***NO se puede retroceder*** entre preguntas. Una vez que haya aceptado la entrega de esta pregunta, lo escrito se considerará *toda* la entrega, sin posibilidad de revisión. Asegurarse de haber resuelto todo a satisfacción ANTES de avanzar.

```
cantSeq :: TCommand -> Int
cantSeq = foldTC (\d -> 0)
              (\a -> 0)
              (\p -> 0)
              (\r1 r2 -> 1 + r1 + r2)
```

Comentario:

NOTA: B

OBSERVACIONES:

* Muy bien

Pregunta **4**

Finalizado

Puntúa 2,00
sobre 10,00

Definir sin usar recursión explícita la función `cantSeqsALaIzqDeSeq :: TCommand -> Int`, que dado un programa de gráficos de tortuga, describe la cantidad total de constructores (`:#:`) del mismo que están en algún argumento izquierdo de un (`:#:`).

ATENCIÓN: recordar que ***NO se puede retroceder*** entre preguntas. Una vez que haya aceptado la entrega de esta pregunta, lo escrito se considerará *toda* la entrega, sin posibilidad de revisión. Asegurarse de haber resuelto todo a satisfacción ANTES de avanzar.

```
cantSeqsALaIzqDeSeq :: TCommand -> Int
cantSeqsALaIzqDeSeq = foldTC (\d -> 0)
                              (\a -> 0)
                              (\p -> 0)
                              (\r1 r2 -> r1)
```

Comentario:

NOTA: M

OBSERVACIONES:

* No. Así definida, la función siempre da 0... Se requería utilizar recursión primitiva...

Pregunta **5**

Finalizado

Puntúa 2,00
sobre 10,00

Definir sin usar recursión explícita la función **assocDer :: TCommand -> TCommand**, que dado un programa de gráficos de tortuga, describe uno equivalente que no tiene argumentos a la izquierda de un **(:#:)** que estén formados por **(:#:)**.

SUGERENCIA: Considerar definir una función auxiliar para el caso inductivo.

ATENCIÓN: recordar que **NO se puede retroceder** entre preguntas. Una vez que haya aceptado la entrega de esta pregunta, lo escrito se considerará *toda* la entrega, sin posibilidad de revisión. Asegurarse de haber resuelto todo a satisfacción ANTES de avanzar.

```
simpSeq :: TCommand -> TCommand -> TCommand
simpSeq (Go d) t = ((Go d):#:t)
simpSeq (Turn a) t = ((Turn a):#:t)
simpSeq (GrabPen p) t = ((GrabPen p):#:t)
simpSeq (t1:#:t2) t = t1:#: (t:#:t2)
```

Comentario:

NOTA: R-

OBSERVACIONES:

* El último caso de simpSeq invierte el orden entre t2 y t. Además, no contempla que t2 puede tener otros constructores (:#:) (se requería también una función auxiliar recursiva)

Pregunta **6**

Finalizado

Puntúa 2,00
sobre 10,00

Definir la función **scaleTC :: TCommand -> Float -> TCommand**, expresada **como** recursión estructural implícita sobre **TCommand**. Esta función escala cada comando **Go** de su argumento según un factor dado. Por ejemplo, **scaleTC(Go 10 :#: Go 30) 2 = (Go 20 :#: Go 60)**.

ATENCIÓN: recordar que **NO se puede retroceder** entre preguntas. Una vez que haya aceptado la entrega de esta pregunta, lo escrito se considerará *toda* la entrega, sin posibilidad de revisión. Asegurarse de haber resuelto todo a satisfacción ANTES de avanzar.

```
scaleTC :: TCommand -> Float -> TCommand
scaleTC = foldTC (\d n -> (Go d*n))
              (\a n -> (Turn a))
              (\p n -> (GrabPen p))
              (\r1 r2 n -> (r1:#:r2))
```

Comentario:

NOTA: R-

OBSERVACIONES:

* Los argumentos r1 y r2 NO son TCommands, sino funciones.

Pregunta **7**

Finalizado

Puntúa 2,00
sobre 10,00

Dar una versión de la función **cantSeqs** dada a continuación, sin utilizar recursión explícita

```
cantSeqs (c1 :#: c2) = let (cs1, csis1) = cantSeqs c1
                        (cs2, csis2) = cantSeqs c2
                        in (1+cs1+cs2, cs1+csis1+csis2)

cantSeqs c = (0,0)
```

ATENCIÓN: recordar que **NO se puede retroceder** entre preguntas. Una vez que haya aceptado la entrega de esta pregunta, lo escrito se considerará *toda* la entrega, sin posibilidad de revisión. Asegurarse de haber resuelto todo a satisfacción ANTES de avanzar.

```
cantSeqs :: TCommand -> (Int,Int)
cantSeqs = foldTC (\d -> (0,0))
                (\a -> (0,0))
                (\p -> (0,0))
                (\r1 r2 -> sumarSeqs 1 (fst r1) r1 r2)
```

Comentario:

NOTA: B-

OBSERVACIONES:

* ¡Qué complicada que la hiciste a sumarSeqs! Podía ser simplemente sumarSeqs (x,y) (w,z) = (1+x+w, x+y+z).

Pregunta **8**

Finalizado

Puntúa 2,00
sobre 10,00

Demostrar que la versión dada de **cantSeqs** en el ejercicio anterior, efectivamente lo es.

ATENCIÓN: recordar que **NO se puede retroceder** entre preguntas. Una vez que haya aceptado la entrega de esta pregunta, lo escrito se considerará *toda* la entrega, sin posibilidad de revisión. Asegurarse de haber resuelto todo a satisfacción ANTES de avanzar.

```
para todo c:: TCommand. cantSeqs (c1 :#: c2) = let (cs1, csis1) = cantSeqs c1
                                                (cs2, csis2) = cantSeqs c2
                                                in (1+cs1+cs2, cs1+csis1+csis2)

cantSeqs c = (0,0)
```

Comentario:

NOTA (Planteo, CB, CI): R--, B, R-

OBSERVACIONES:

- * La propiedad a plantear era (cantSeqs = cantSeqs'), siendo la primera explícita, y la segunda la implícita. Así como está, no dice eso, sino algo confuso.
- * En el planteo del caso inductivo aparecen variables que no están ligadas (parece copia de la definición sintáctica de la definición recursiva explícita)
- * Ese planteo muestra falta de comprensión de conceptos y trivializa el desarrollo del caso inductivo.